

Assignment 3: System Design & Architecture Documentation

Nurshapagat Shapay, Akezhhan Kumar CSE-2501M

Task 1. Non-Functional Requirements & Trade-offs

1.1. Define your NFR targets

Quality attribute	Target (numeric)	Measurement method	Justification
Latency	P95 < 3000ms (dashboard load)	Load testing, k6	As we described in the 1 assignment, 4G mobile users needs fast loading time when they open the app or when adding/checking expenses
Availability	99% uptime	Uptime monitoring, UptimeRobot	Users calculating the debts in real time and instantly, if the app is down then someone needs to pay, they lose trust
Throughput	300 RPS peak	Benchmark tool, k6	Overall concurrent users can reach 300, for example a big group who trip or any events
Consistency	Strong	Per feature, ACID transaction tests	As we said before, a wrong balance destroys our trust. Because users handles real money, therefore, each expenses should be atomic
Data durability	RPO < 24 hour	DR test, Backup restore	Users store monthly expense history, and losing that data makes PayBack useless. So, we should ensure daily backups

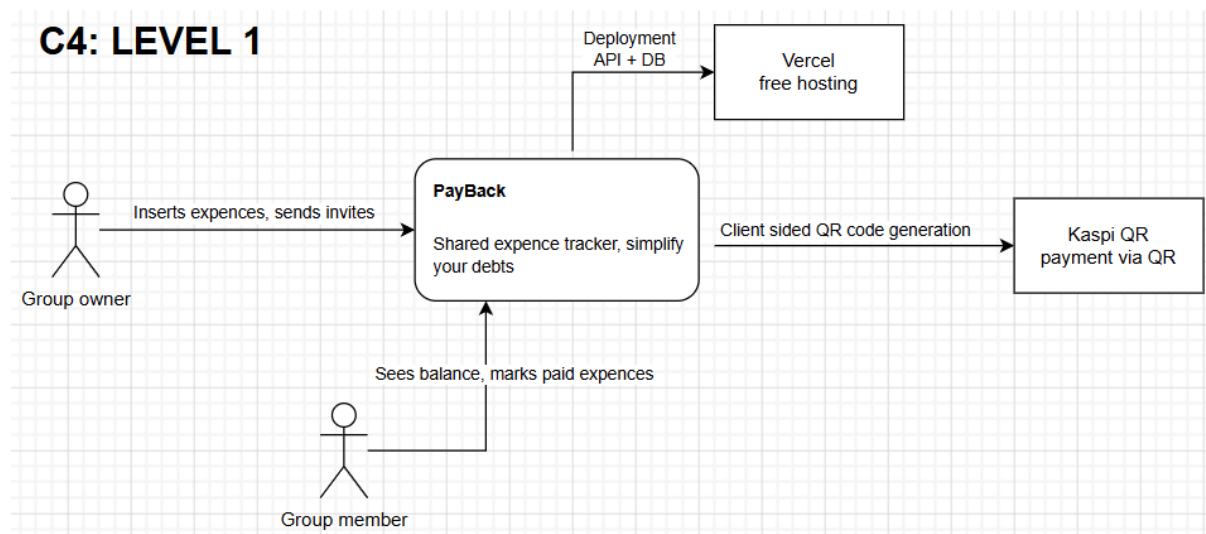
1.2 Identify your primary trade-off

Trade-off axis	What you optimize FOR	What are you willing to give up
Latency vs Throughput	<i>We prefer optimize throughput, because supporting 300 concurrent users matters over than ensure few millisecond for single request</i>	<i>We give up with higher latency under peak load</i>

Consistency vs Availability	<i>We prefer optimized consistency, because each expense should be atomic with 0 partial writes allowed. We would rather make users wait 50 ms than show a wrong debt information.</i>	<i>We give up availability, because we block the new request during database transactions. It ensure a right balance every time</i>
Cost vs Performance	<i>We prefer to optimize cost, because it's a MVP developed by 2 people with zero budget. The free tier hosting on Vercel and Railway is enough for us.</i>	<i>We give up with performance, because we prefer the free tier hosting for MVP projects. We can improve performance in the future, when the product grows</i>

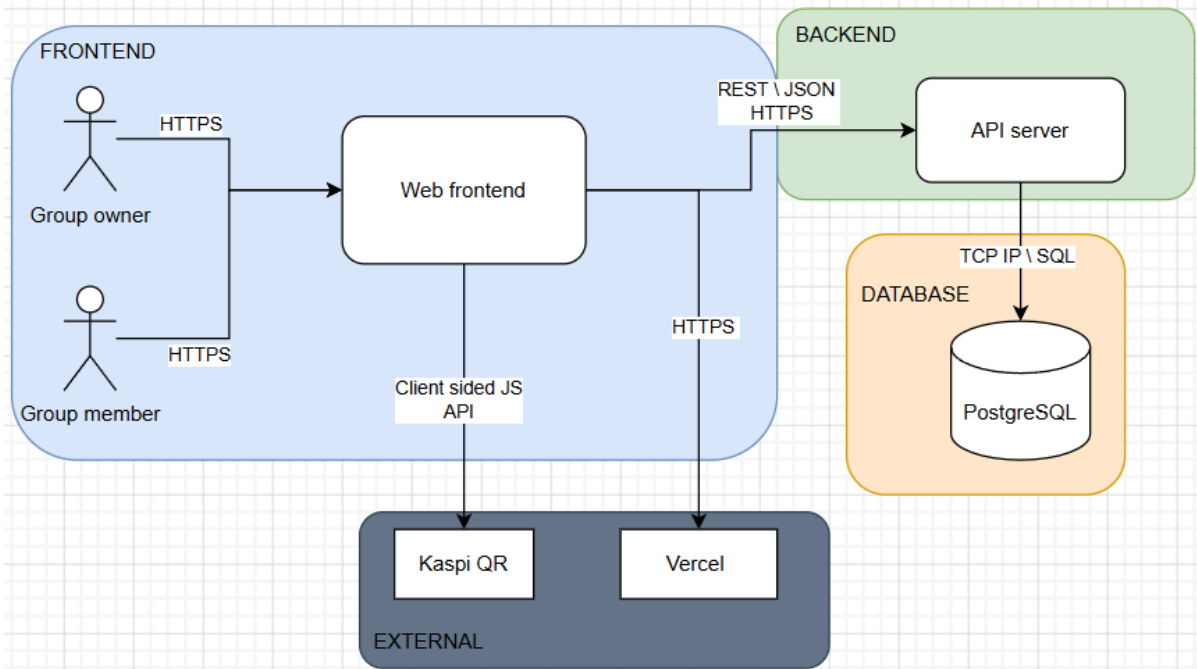
Task 2. High-Level Architecture Diagram (C4 Model - L1 & L2)

Level 1 - System Context diagram

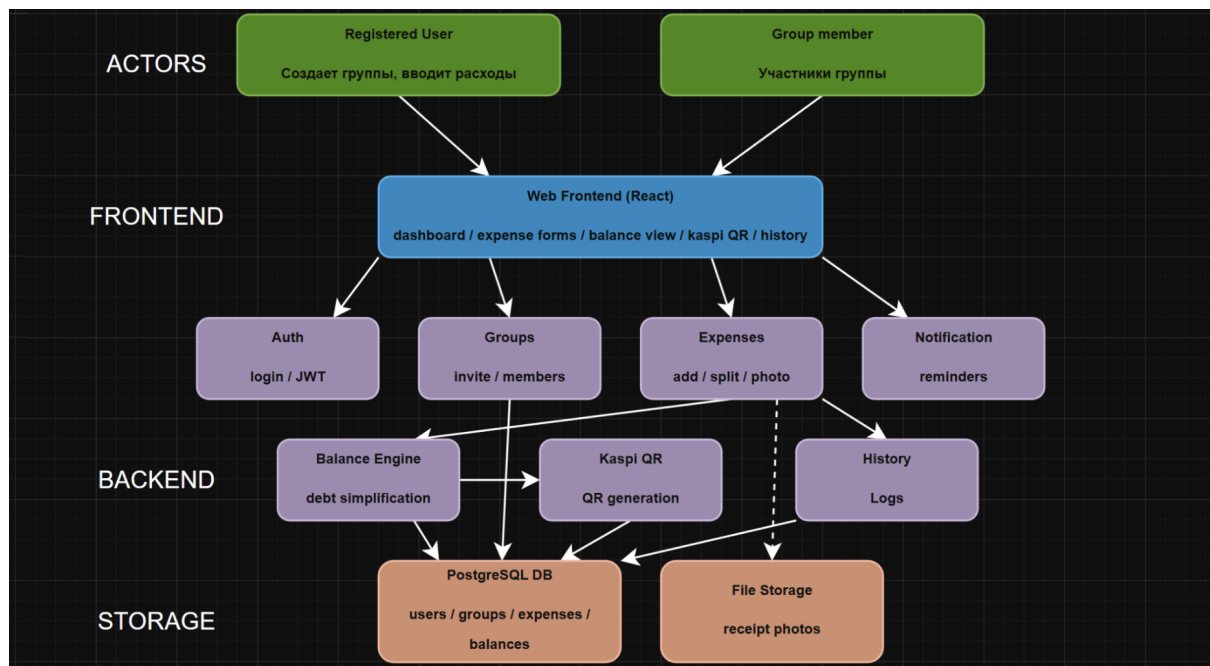


Level 2 - Container diagram

C4: LEVEL 2



Level 3 – Component Diagram



Task 3. Architecture Decision Record (ADR)

ADR template (fill in each section)

ADR-001	Use PostgreSQL over MongoDB
Status	accepted

Date	18 April 2026
Context	We need a database that handles atomic writes with multiple tables at once, because our tables are related to each other. Our database contains tables like: users, groups, balances, expenses, splits
Decision	We choose a PostgreSQL as a primary database
Rationale	Our data are relational, so: 1. JOINS are required indexes by querying for fast reading, as we described in latency NFR 2. ACID should perform all 4 transactions, as we described in consistency NFR 3. Railway provides auto daily backups, as we described in durability NFR 4. Horizontal scalability need when stateless backend can scale independently
Alternatives rejected	Mongodb not allows multi document transactions in the free tier SQLite can't handle with 300 concurrent users
Consequences	Easier: JOINS, built transactions, auto backup Harder: every simple changes needs migrations in the tables Risk: under high load can influence to free tier Railway limits on multiple connections to the database

Task 4. Detailed Component Design

Web Frontend Component

Component name	Web Frontend
Purpose	Shows UI and takes user inputs
Responsibilities	1. Display dashboard with expenses 2. Generate KaspiQR, on client side 3. Send HTTPS request to the API server 4. Show real time balance
Inputs	User data: forms, button clicks, any other frontend interactions. (<code>{amount: 7500, category: "food", member: ["Nurshapagat, Akezhan"]}</code>)
Outputs	Rendered UI, HTTP requests from API server. (<code>POST /expenses{group_id, amount}</code>)
Dependencies	API server (sends request to) KaspiQR (uses to generate QRs)
NFR sensitivity	Latency: dashboard load time must be P95 < 3000ms

API Server Component

Component name	API server
Purpose	Handle business logic and database operations
Responsibilities	1. Validate requests 2. Validate JWT tokens 3. Process HTTPS requests from Web Frontend 4. Operate with database 5. Run debt simplification algorithm
Inputs	HTTPS request from Web Frontend (POST /expenses{group_id, paid_by, amount})
Outputs	JSON responses (201 {expense_id, splits[]}, 400{error:"invalid data"})
Dependencies	PostgreSQL (operate data) Web Frontend (receives responses from)
NFR sensitivity	Consistency: all expenses must be atomic, writing in database crucial Throughput: must handle 300 users at the same time

PostgreSQL Component

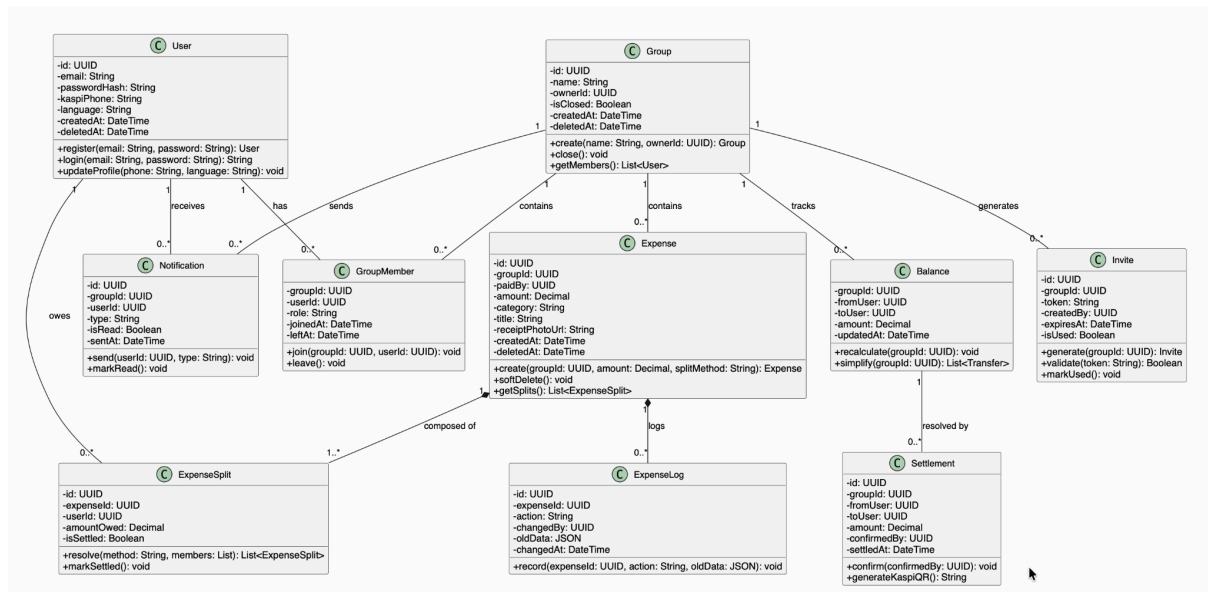
Component name	PostgreSQL
Purpose	Stores and protect all necessary data securely
Responsibilities	1. Stores all necessary data: users, groups, expense, balances 2. Provides foreign key constraints between tables 3. Allows ACID by running atomic transactions 4. We can make auto backups every day with Railway
Inputs	sql queries from api server INSERT INTO expenses values (...)
Outputs	the output from query results {expense id: "uuid", created_id: "...", } error case when makes foreign key violation
Dependencies	It calls by API server, nobody else doesn't touch it directly
NFR sensitivity	Consistency by ACID transaction and durability by Railway backups

Kaspi QR Component

Component name	Kaspi QR
Purpose	Generating a QR code for users who should pay to another user instantly directly in the app without leaving
Responsibilities	1. You can share a phone number with amount 2. Generate qr code that directly redirects to app where you can pay a debt 3. If it doesn't working, just shows a phone number and amount as a text
Inputs	directly in the app {kaspi_phone: "875775758", amount: "22500"}
Outputs	generating a qr code image in the screen Fallback (if qr code not accessible): 875775758: 22500 kzt
Dependencies	It calls by only Web Frontend, it works in client side, so it doesn't depend with any external api and server
NFR sensitivity	Availability by working in client side even when server overloaded and cost because all free, it is the best variant for broke people like us

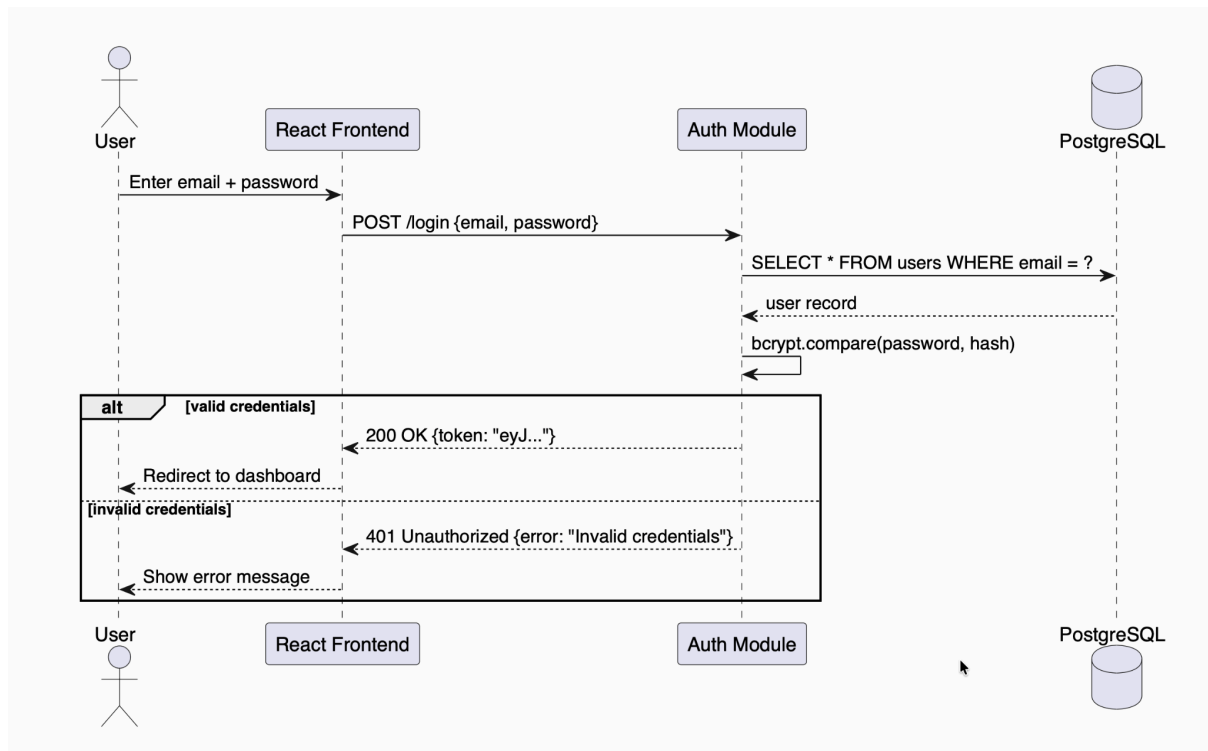
Task 5. UML Diagrams

Task 5.1 - Class Diagram

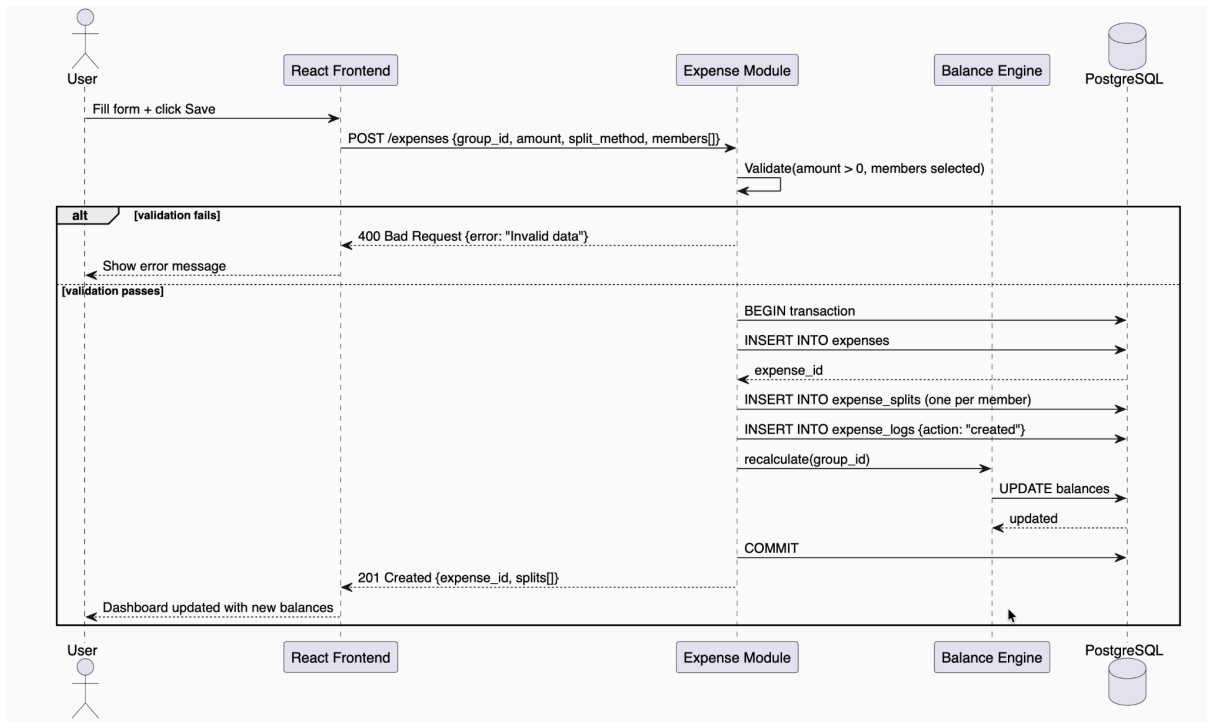


Task 5.2 - Sequence Diagram

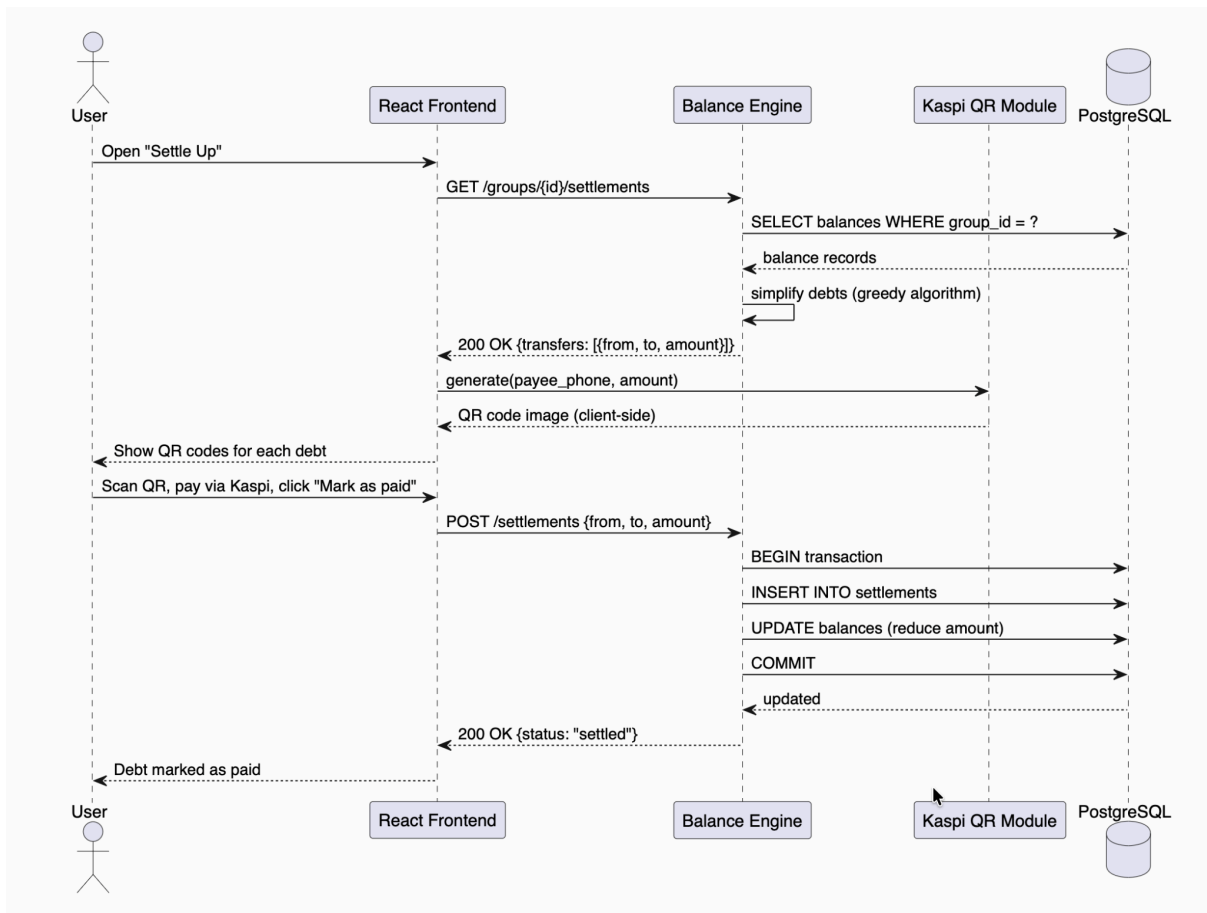
1. User Login



2. Add Expense



3. Debt Simplification



Task 5.3 - Entity-Relationship Diagram

